조대표님, 맨션된 내용(현재 문제 **SSOT DB 1**개 **+ exams.json +** 로컬 **localStorage**) 구조는 **"**문제풀이**"**까지는 완벽한데, 말씀하신 **"개인화 성적관리"**는 성격이 다른 데이터라서 별도 **DB(**또는 별도 데이터 레이어**)** 가 반드시 필요합니다. 핵심은 문제은행**(**콘텐츠**)** 과 풀이이력**/**성적**(**사용자 이벤트**)** 를 분리하는 것입니다.

1) 지금 구조의 "문제" 정의

현재 Notion DB는 문항 원본**(SSOT)** 입니다.

- 성격: 비교적 정적(가끔 수정/추가)
- 파이프라인: Notion DB → `exams.json` → 프론트 소비
- 여기까지는 Phase 1에서 최적입니다.

반면 성적/통계는

- 성격: 초고빈도 이벤트**(**매 클릭**/**매 문항**)** **+** 사용자별 **+** 시간축 데이터
- 즉, 같은 DB에 얹으면 스키마도 복잡해지고 운영/성능/정합성이 같이 흔들립니다.

2) "개인화 성적관리"를 위해 추가로 필요한 데이터(최소 단위)

통계를 만들려면 최소 아래 3종이 있어야 합니다.

1. 사용자**(**프로필**/**설정**)**
- 시험일, 이름, 목표, 하루 목표 문항수 등
1. 풀이 이벤트**(Attempt;** 문항 단위 기록**)**
- 누가, 언제, 어떤 문제를, 무엇을 선택했고, 정답/오답이었는지
- 이것이 통계의 원장(ledger)입니다.
1. 세션**(**학습 단위 묶음**; Session)**
- "오늘 랜덤 30문제", "31회 법령 1~40" 같이 묶어야 "회차별/일자별 성적"이 깔끔해집니다.

3) 선과장(오너) 권고안**: Phase 1**에서 가장 안전한 **"**추가 **DB"** 설계 **2**가지 옵션

조대표님은 "추가 DB가 필요"라고 하셨으니, Notion SSOT 관점에서 옵션을 두 개로 제시하겠습니다.

옵션 **A (**권장**): Notion**에 **"**성적 원장 **DB"**를 따로 만든다 **(SSOT**형**)**

추가 데이터소스 **2**개면 충분합니다.

(A-1) `GEP Users` (사용자**)**

- userId (title) : 예 `USER_0001` 또는 휴대폰 해시

- 이름 (text)
- 시험일 (date)
- 생성일 (created_time)

(A-2) `GEP Attempts` (풀이 원장; 가장 중요)

- attemptId (title) : 예 `ATT_20260221_000001`
- user (relation → GEP Users)
- questionId (text) : `IBEX_31_003_T1_S01_003`
- timestamp (date; datetime)
- selectedAnswer (number)
- correctAnswer (number) 또는 isCorrect (checkbox)
- round (number)
- subject (select)
- subSubject (select)
- sessionId (text) 또는 relation(선택)

장점

- 개인화 성적이 Notion에 쌓이므로 "조대표님 SSOT 원칙"에 정합.
- 나중에 Phase 2(서버/유료DB)로 옮겨도 데이터 마이그레이션이 명확.

단점

- Phase 1에서 Notion API로 "매 문항 저장"은 속도/레이트리밋 리스크가 큽니다.
  - 그래서 "실시간 전송"이 아니라 배치 업로드(하루 **1**회/수동 버튼) 로 설계해야 안정적입니다.

결론: "DB는 Notion에 만들되", 앱에서는 일단 **localStorage**에 쌓고, 일정 단위로 Notion에 적재하는 하이브리드가 Phase 1 최적입니다.

옵션 **B: Phase 1**은 **localStorage**만으로 통계를 끝까지 간다 (로컬형)

추가 Notion DB 없이도 통계는 만들 수 있습니다.

- 이미 `answers`를 저장 중이므로, 여기에
  - attempt 배열(시간 포함)
  - dayIndex(YYYY-MM-DD)
  - 누적 정답수/풀이수 캐시

를 붙이면 됩니다.

장점

- 개발/속도/비용 최강. 오프라인도 강함.

단점

- 기기 변경/데이터 백업/공유가 약함.

- 조대표님 관점의 “SSOT=Notion” 원칙과는 거리가 생김.

4) “PHASE 1-5 통계 로직 완전히 정리하기” 관점에서의 정답 구조(통계 계산 방식)

통계는 DB보다 원장(**Attempts**) 기반으로 계산하는 게 정답입니다.

- 일일 통계: `timestamp`를 날짜로 버킷팅해서
  - 오늘 풀이수, 오늘 정답률
- 회차/과목/세부과목 통계: Attempts를 해당 차원으로 group
  - (round, subject, subSubject)별 풀이수/정답률
- 진도(**Progress**): “필터 조건에서 unique questionId 중 answered 개수”
- 오답노트: isCorrect=false인 attempt의 questionId unique 목록

여기서 중요한 정책 2개:

1. 중복풀이를 어떻게 처리할지(최근 1회만? 누적 모두?)
2. “정답률”을 문제 기준으로 볼지, 시도 기준으로 볼지

이 2개가 확정되면 DB 스키마도 고정됩니다.

5) 조대표님께 확인 질문 (3개만)

추가 DB를 “딱 맞게” 제안하려면 아래 3개만 답 주시면 됩니다.

1. 사용자 범위: 1인(조대표님만) 인가요, 아니면 다수 사용자(익명 포함) 까지 목표인가요?
2. 데이터 보존: 성적/이력은 기기 변경해도 유지되어야 하나요?
3. 통계 기준: 오답노트/정답률은 최근 1회 기준이 맞나요, 아니면 시도 누적 기준이 맞나요?

위 3개 답이 오면, 선과장 오너 의견으로 “추가 DB는 1개로 충분/2개가 최적”까지 결론을 딱 내려드리겠습니다.

조대표의견
1. 유료서비스를 전제로 합니다. 유저가 수백명이 될수도 있고 향후 다른 과목까지 확장하면 수천명이 될 수 있습니다 2. 데이터 보존은 월구독료 서비스 이므로 유료 회원에게 대해서 제공하고 돈을 내지 않으면 학습데이터는 초기화 됩니다(사실은 1년 정도 개인별 성적데이터를 보유하고 있을 생각이지만 대외적으로는 매월 유료사용자의 데이터는 보존 합니다 통상 시험 준비를 위해서 6개월에서 10개월 정도 유효할 것으로 추정합니다) 3. 통계 기준은 시도누적 기준입니다. 총 1080문제 중에서 처음 풀이 했을때 1000문제가 틀렸다고 하면 틀린문제 풀기를 통해서 그 횟수를 게속 줄여가야 합니다. 특정 문제를 내가 6개월 똥안 30번 풀었는데 그중에서 몇번 정답을 맞추었다도 나와야 합니다

선과장 답변

조대표님 답변 3개를 기준으로 결론부터 말씀드리면, **Notion**에 **"**성적관리 **DB"**를 추가하는 방식은 **Phase 1**에는 임시로 가능하지만**,** 유료서비스**(**수백**~**수천 유저**)** 전제에서는 운영 **SSOT**로 쓰기 어렵습니다**.**

오너(선과장) 의견은 **"문제은행(콘텐츠) = Notion SSOT 유지" + "학습데이터(Attempt 원장) = 유료 DB로 분리"**가 정답 구조입니다.

1) 요구사항을 구조로 번역 (핵심 제약 3개)

1. 규모: 유저 수백→수천, 과목 확장. 즉 "고빈도 쓰기 + 대량 조회"가 필수.
2. 보존 정책: 구독 중에는 유지, 해지 시 초기화(대외). 실제로는 6~12개월 보관 가능성.
3. 통계 기준: 시도 누적**(Attempt** 누적**)**.
    - 한 문제를 6개월간 30번 풀면 "30회 시도, 정답 몇 회, 최근 정답/오답, 정답률"이 나와야 함.
    - "처음 1000개 오답 → 오답 줄여가기"도 누적 시도 기반으로 계산.

이 3개는 이벤트 원장**(Attempts ledger)** 없이는 정확하게 못 만듭니다.

2) 반드시 분리해야 하는 DB 2계층 (오너 의견)

A. 콘텐츠 계층 (Notion 유지)

- 문제 **DB(**현 **SSOT)**: 지금처럼 Notion에 둡니다.
- 역할: 편집/검수/추가/정정에 최적.
- 배포: Notion → `exams.json` (또는 subject별 json)로 계속.

B. 학습데이터 계층 (별도 유료 DB 필수)

- 역할: "매 문항 클릭마다 저장", "유저별/기간별 통계 쿼리"의 주력 저장소.
- Notion으로 하면:
    - API rate limit + 비용 + 조회/집계 한계로 규모에서 터집니다**.**
    - 특히 Attempt 누적은 데이터가 기하급수로 커집니다.

3) 추가로 필요한 "DB(테이블) 최소 4종" — 이게 개인화 성적관리의 뼈대

아래는 벤더/DB종류와 무관하게 논리 모델입니다.

1. **users**
- user_id, created_at
- (옵션) 이름/휴대폰/마케팅동의 등
2. **subscriptions**
- user_id, plan, status(active/canceled), started_at, ended_at
- *"구독 중이면 데이터 유지"*의 판정 근거
3. **attempts (**가장 중요**:** 원장**)**

- attempt_id
- user_id
- question_id (IBEX_ID 같은 불변키)
- exam_version (exams.json version)
- selected_answer
- correct_answer (또는 is_correct)
- attempted_at (timestamp)
- (옵션) device_id, session_id, mode(랜덤/오답/전체), latency 등

4. **question_stats (집계 캐시: 성능용)**

- user_id + question_id 복합키
- total_attempts
- correct_attempts
- wrong_attempts
- last_attempt_at
- last_is_correct
- first_correct_at (선택)
- streak 같은 지표(선택)

왜 **"attempts + stats"**를 같이 두나**?**

- attempts는 "진실의 원장"이라서 절대 필요합니다.
- stats는 매번 attempts를 전수 집계하면 느리니, 앱 경험을 위해 업데이트형 캐시가 필요합니다.

4) 조대표님이 말한 통계가 어떻게 계산되는지 (시도누적 기준)

조대표님 예시를 그대로 충족시키려면 지표 정의는 이렇게 고정됩니다.

- "총 1080문제 중 오답 줄여가기"
  - 오답문제 수 = `question_stats`에서 `wrong_attempts > 0 AND correct_attempts = 0` 같은 정의(또는 "최근 10회 중 오답" 등 정책 선택)
- "특정 문제 30번 중 정답 몇 번"
  - `question_stats.total_attempts = 30`
  - `question_stats.correct_attempts = X`
  - 정답률 = X/30
- "6~10개월 유효"
  - 시도 로그(attempts)는 보통 1년 보관해도 되고,
  - 해지 시 대외적으로는 초기화이므로:
    1. 즉시 삭제(강경) 또는
    2. **soft delete + 1년 TTL** 후 물리삭제(추천: 환불/분쟁/재구독 복구 대비)

    - 정책은 DB 설계에 반영 가능 (예: deleted_at 컬럼)

5) “추가적인 디비”를 Notion 관점에서 어떻게 가져갈까 (현실적인 하이브리드)

오너 의견: Notion은 여기서 **2**가지 역할만 맡기는 게 가장 깔끔합니다.

1. 콘텐츠 **SSOT(**문제은행**)**
2. 운영**/**리포트 대시보드**(**집계 결과를 요약해서 보는 곳**)**
    - 단, 원장을 Notion에 다 넣지는 않습니다.
    - 예: “일간 활성 사용자 수”, “과목별 평균 정답률”, “유료 전환율” 같은 운영지표 요약만.

6) 지금 당장 “PHASE 1-5 통계 로직”에서 확정해야 할 결정 5개 (스키마가 흔들리는 지점)

1. **question_id**의 불변성: IBEX_ID를 영구 키로 확정(이미 좋은 상태)
2. 정답 변경 시 처리: 문제 정정으로 correct_answer가 바뀌면 과거 attempt는 재평가할지
3. 오답노트 정의: “한 번이라도 틀림” vs “최근 N회 기준” vs “정답 맞출 때까지”
4. 구독 해지 데이터 처리: 즉시 삭제 vs 1년 보관(soft delete) + 외부 노출만 초기화
5. 과목 확장 구조: subject/subSubject를 데이터에서 추출할지, 별도 테이블로 관리할지

조대표님께 2가지만 추가로 확인하면, 제가 “추가 DB 설계(최소 테이블 + 핵심 인덱스 + 통계 쿼리 기준)”를 바로 확정해서 드릴 수 있습니다.

1. 동일 계정의 멀티 디바이스를 지원해야 하나요? (폰/태블릿/PC 동기화)

2. “해지 시 초기화”를 즉시 물리삭제로 갈까요, 아니면 **1**년 보관**(**재구독 시 복구 가능**)** 으로 갈까요?

조대표 의견
1. 동일계정의 멀티 디바이스를 지원해야 합니다 2. 해지시 1년 보관 후 삭제로 합니다 왜냐면 시험준비 했다가 떨어지고 다시 볼 사람도 있습니다

조대표님 확인했습니다. 멀티 디바이스 지원 **+** 해지 후 **1**년 보관**(**재구독 복구 가능**)** 이면, 오너(선과장) 관점에서 데이터 구조는 사실상 **“계정 기반 서버 저장 + 원장(Attempt) + 집계 캐시”**로 고정됩니다. 로컬(localStorage)은 오프라인 버퍼**/**캐시까지만 허용하는 게 맞습니다.

1) 최종 아키텍처 결론 (오너 의견)

- **Notion**: 문제은행 SSOT(지금처럼) + 운영/리포트(요약치) 용도
- 유료 **DB(**서버**)**: 유저/구독/학습데이터 SSOT
  - 멀티 디바이스는 "서버에 계정별 데이터가 있어야" 성립합니다.
  - 해지 후 1년 보관도 "서버 보관 + 접근 차단" 정책으로 푸는 게 정석입니다.

즉, "추가 DB"는 Notion DB가 아니라 서비스 **DB(Postgres** 등**)** 쪽에 추가가 핵심입니다.

2) 테이블 설계(최소 확정 5종) — 멀티 디바이스/1년 보관 반영

(1) users

- `user_id` (PK)
- `email` 또는 `phone` (unique, 로그인 ID)
- `created_at`

(2) devices (멀티 디바이스 지원의 핵심)

- `device_id` (PK)
- `user_id` (FK)
- `created_at`, `last_seen_at`
- (옵션) `device_name`, `platform`

(3) subscriptions

- `subscription_id` (PK)
- `user_id` (FK)
- `status` = active / canceled / past_due …
- `current_period_start`, `current_period_end`
- `canceled_at`

(4) attempts (원장; 누적 통계의 진실)

- `attempt_id` (PK)
- `user_id` (FK)
- `question_id` (IBEX_ID)
- `attempted_at` (timestamp)
- `selected_answer` (int)
- `correct_answer` (int) 또는 `is_correct` (bool)
- `exam_version` (예: exams.json version)
- `device_id` (FK, optional)

- (옵션) `mode` (random / wrong-only / sequential)

(5) question_stats (집계 캐시; 속도 필수)

- PK: (`user_id`, `question_id`)
- `total_attempts`
- `correct_attempts`
- `wrong_attempts`
- `last_attempt_at`
- `last_is_correct`
- (옵션) `first_correct_at`

  attempts는 "감사/복구/정밀 통계"를 위해 남기고, 화면/대시보드는 stats를 주로 조회합니다.

3) "해지 시 초기화(대외)" + "1년 보관(내부)"를 동시에 만족하는 정책

오너 의견으로는 **물리삭제가 아니라 "접근권한 차단 + 보관 타이머"**가 맞습니다.

- 구독이 **active**: 모든 API 정상
- 구독이 **canceled**:
  - 앱에서는 "데이터 초기화된 것처럼" 보이게 처리(새 attempts 기록 금지, stats 조회 금지)
  - DB에는 그대로 보관
- **canceled_at + 1**년이 지나면:
  - 백엔드 배치 작업으로 `attempts`, `question_stats`를 물리 삭제
  - (권장) 삭제 전 "요약만 남길지"는 선택인데, 조대표님 정책상 "초기화"를 강조하니 완전 삭제가 더 일관합니다.

이렇게 하면 "재구독하면 바로 복구"도 가능하고, 대외 커뮤니케이션도 단순해집니다.

4) 멀티 디바이스에서 반드시 필요한 동기화 규칙 3개

1. 정답 저장의 단일 진실은 서버
- 디바이스 A에서 푼 내용이 디바이스 B에서 즉시 보여야 함.
2. 로컬은 '오프라인 큐'로만 사용
- 네트워크 끊기면 attempts를 로컬 큐에 적재
- 온라인 복귀 시 서버에 업로드 후 큐 비우기
3. 충돌 해결은 **"원장 append-only"**라서 단순
- attempts는 수정이 아니라 "추가"라 충돌이 거의 없습니다.
- stats는 서버에서 원장 기반으로 증가 업데이트.

5) 조대표님께 이제 “딱 1개”만 결정 요청 (DB 설계 완결 게이트)

로그인/결제 연동을 전제로 계정 식별자를 무엇으로 갈지만 확정하면 스키마가 완전히 고정됩니다.

- A안: 휴대폰 번호 기반 로그인(OTP)
- B안: 이메일 기반 로그인(매직링크)
- C안: 소셜 로그인(카카오/구글)

조대표님은 어떤 쪽이 제품에 맞나요? (하나만 선택해주시면, 그 전제로 “API 엔드포인트 최소 세트 + 통계 화면에 필요한 쿼리”까지 다음 답에서 확정해드리겠습니다.)

조대표님, 카카오/네이버 간편로그인으로 확정이면 오너(선과장) 의견은 이렇게 정리됩니다. 핵심은 “로그인 공급자(OAuth) 계정”을 서비스의 유저(User)와 분리해서, 공급자가 늘어나도(카카오→네이버→구글) 데이터 모델이 흔들리지 않게 만드는 것입니다.

1) 인증(로그인) DB 구조 — 최소 3테이블이 정석

(1) users (서비스 유저의 본체)

- `user_id` (PK, 내부 UUID)
- `created_at`
- `status` (active / blocked 등)

(2) oauth_accounts (간편로그인 연결 테이블) 필수

- `provider` (kakao / naver / google / apple …)
- `provider_user_id` (공급자가 주는 고유 ID, 문자열)
- `user_id` (FK → users)
- `created_at`, `last_login_at`
- unique 인덱스: `(provider, provider_user_id)`

  → 같은 카카오 계정이 두 유저로 중복 연결되는 사고를 원천 차단

(3) sessions (로그인 세션)

- `session_id` (PK)
- `user_id` (FK)
- `refresh_token_hash` (원문 저장 금지)

- `created_at`, `expires_at`, `revoked_at`
- (옵션) `device_id` 연결

이 3개가 있으면 “카카오/네이버 둘 다 지원”, “멀티 디바이스”, “로그아웃/토큰폐기”가 모두 깔끔해집니다.

2) 멀티 디바이스까지 포함한 “권장 최소 세트” (앞서 합의한 학습DB와 결합)

조대표님 요구사항(멀티 디바이스, 누적 시도, 해지 1년 보관)을 붙이면 서비스 DB 최소 테이블은 아래 7개로 고정됩니다.

1. `users`
2. `oauth_accounts`
3. `sessions`
4. `devices`
5. `subscriptions`
6. `attempts` (원장)
7. `question_stats` (캐시)

3) “해지 시 초기화처럼 보이게” 만드는 가장 안전한 구현 규칙

- 서버는 데이터 삭제를 즉시 하지 않습니다.
- API 게이트에서 `subscription.status !== active`이면:
  - attempts 기록 API 차단
  - stats 조회 API 차단 또는 “0으로 응답”
- 재구독 시 즉시 복구: 차단만 풀면 stats/attempts가 그대로 살아있음
- **canceled_at + 1**년 지난 유저만 배치로 물리삭제

이 방식이 “대외적으로는 매월 보존(구독중 유지)”과 “내부적으로는 1년 보관”을 동시에 만족합니다.

4) 조대표님께 마지막으로 2가지만 확정 요청 (실행 설계로 넘어가기 위한 게이트)

간편로그인은 “카카오/네이버”로 정해졌으니, 이제 운영 룰 2개만 확정하면 백엔드/프론트 명세가 완성됩니다.

1. 계정 통합 정책: 사용자가 카카오로 가입 후, 나중에 네이버로도 로그인하면
   - A안: 자동으로 같은 계정으로 합치지 않고, “계정 연결” 버튼을 통해서만 연결
   - B안: 이메일/휴대폰이 같으면 자동 병합(사고율 높아서 비권장)

2. 게스트(비로그인) 허용 여부: 지금처럼 “익명학습”을 남길까요, 아니면 유료 전제라서 처음부터 로그인 강제할까요?

위 2개만 답 주시면, 다음 단계로 **필수 API 엔드포인트 목록(로그인/구독/attempt 업로드/stats 조회) + 프론트 변경 포인트(examStore에서 서버 동기화)**를 오너 의견으로 바로 확정해서 드리겠습니다.

조대표님, 확정(1 A안, 2 게스트 통계 미적용) 기준으로 데이터/DB 구조와 API가 이제 완전히 고정됩니다. 오너(선과장) 의견으로 “이대로 가면 된다” 수준까지 내려서 드리겠습니다.

1) 정책 확정 요약(SSOT)

- 계정 통합: 자동 병합 금지. “계정 연결(merge/link)”은 사용자 액션으로만 허용(A안).
- 게스트: 문제 풀기는 허용하되, 서버에 **attempts/stat** 저장 안 함.
  - 즉, 게스트는 “기능 체험용”이고, 성적/통계는 회원 기능.

2) DB 스키마 최종본(간편로그인 + 통계)

인증/계정

- `users(user_id, status, created_at)`
- `oauth_accounts(provider, provider_user_id, user_id, created_at, last_login_at)`
  - unique: `(provider, provider_user_id)`
- `sessions(session_id, user_id, refresh_token_hash, created_at, expires_at, revoked_at)`
- `devices(device_id, user_id, created_at, last_seen_at, platform, device_name)` (선택이지만 멀티디바이스면 사실상 권장)

구독/보존

- `subscriptions(subscription_id, user_id, status, current_period_start, current_period_end, canceled_at)`
- 해지 후 1년 보관을 위해 (둘 중 하나 선택)
  - A) `users.deleted_at` (soft delete 타임스탬프)
  - B) `data_retention_until` (예: canceled_at + 1년)

학습 원장/통계

- `attempts(attempt_id, user_id, question_id, attempted_at, selected_answer, correct_answer|is_correct, exam_version, device_id, mode)`

- `question_stats(user_id, question_id, total_attempts, correct_attempts, wrong_attempts, last_attempt_at, last_is_correct, first_correct_at)`

3) API 엔드포인트 "최소 세트"(이대로 만들면 서비스가 닫힙니다)

로그인(OAuth)

- `GET /auth/:provider/start` (카카오/네이버로 리다이렉트)
- `GET /auth/:provider/callback` (code 수신 → 계정 찾기/생성 → 세션 발급)
- `POST /auth/logout` (세션 revoke)

A안(자동 병합 금지) 처리 규칙

- 콜백 시 `(provider, provider_user_id)`가 기존에 있으면 그 user로 로그인
- 없으면 새 **user** 생성 후 oauth_accounts 연결
- "카카오로 가입한 사용자가 네이버로 로그인"하면 새 **user**가 생기는 게 정상
  - 이후 "계정 연결" 화면에서만 link 허용(추후 기능)

구독/권한

- `GET /me` (로그인 상태 + subscription active 여부 + 보존 상태 반환)
- `GET /subscription/status` (또는 /me에 포함)

권한 게이트(중요):

- `subscription.active === false`면 stats/attempts API는 403 또는 빈 데이터로 응답("초기화처럼 보이게")

학습데이터

- `POST /attempts` (회원만)
  - 서버가 attempts insert + question_stats upsert(증가)까지 처리
- `GET /stats/summary` (회원만)
  - 전체 풀이수, 전체 정답률, 오늘 풀이수, 오늘 정답률, 오답문제 수 같은 홈용 요약
- `GET /stats/questions?filter=wrong` (회원만)
  - 오답노트 리스트(문제ID 목록) 등

4) 프론트(현재 **Zustand** 구조)에서 바뀌는 핵심 **2**점

1. 게스트 모드 분기
- `isGuest=true`면 지금처럼 localStorage만 사용(또는 최소 저장)

- 단, “통계 화면”은 비활성/로그인 유도

2. 회원 모드 동기화

- `saveAnswer()` 시점에 서버로 `POST /attempts`
- 네트워크 실패 시 로컬에 “pending queue” 적재 후 재시도
- 홈 요약은 `/stats/summary`에서 가져와 표시

5) 다음 의사결정 1개만 주시면 “개발 지시문” 단계로 넘어갈 수 있습니다

간편로그인/구독을 붙일 때, 초기 결제/구독 구현을 어떤 수준으로 먼저 갈지를 정해야 합니다.

- A) “구독 상태를 일단 서버에서 수동 플래그로 관리” (초기 베타용, 빠름)
- B) “처음부터 결제사 연동까지” (정식 런칭용, 느리지만 정공법)

조대표님은 다음 단계에서 A로 빠르게 MVP를 닫고 갈까요, 아니면 B로 바로 갈까요?

조대표님, **A(구독 상태 수동 플래그로 MVP 닫기)**로 확정이면 “지금 당장 개발 지시문을 내려도 되는” 수준으로 설계를 고정할 수 있습니다. 핵심은 결제 연동 없이도 **`subscription_status`**만으로 모든 권한/통계 흐름을 닫는 것입니다.

1) MVP에서 구독을 “수동 플래그”로 정의 (정의가 곧 스펙)

- 서버에 `subscriptions.status`만 둡니다: `active | inactive | canceled`
- 운영자가 관리자 화면(없으면 임시 스크립트/DB 콘솔)에서 `active`로 바꾸면 즉시 유료회원
- 앱/서버 로직은 결제사가 아니라 **status**만 신뢰합니다

2) 반드시 지켜야 하는 권한 게이트(이게 서비스 정책을 구현)

게스트

- 익명으로 문제풀기 허용
- attempts/stat 서버 저장 금지
- 통계 화면 진입 시 로그인 유도(또는 “회원만 제공”)

로그인했지만 비구독**(inactive/canceled)**

- 서버에서 `/attempts`, `/stats/*`는 차단
- 화면에서는 “데이터가 없는 것처럼” 처리(대외적으로 초기화)
- 단, DB에는 1년 보관을 위해 물리 삭제는 하지 않음(접근만 막음)

구독 **active**

- attempts 저장 + stats 조회 허용
- 멀티 디바이스 동기화 정상 작동

3) 백엔드 최소 구현 순서(고팀장에게 내릴 작업 순서)

1. **DB** 마이그레이션
    - users, oauth_accounts, sessions, subscriptions, attempts, question_stats, devices(권장)
2. **OAuth** 로그인(카카오/네이버)
    - A안 정책: 자동 병합 금지. provider_user_id 기준으로만 계정 매칭
3. **/me API**
    - `isAuthenticated`, `subscriptionStatus`, `retentionUntil(선택)` 반환
4. **attempt** 기록 **API**
    - `POST /attempts`
    - active 아니면 403
    - active면 attempts insert + question_stats upsert(카운터 증가)
5. 통계 요약 **API**
    - `GET /stats/summary`
    - 홈 화면에 필요한 최소 지표만(총 풀이수/정답률/오늘 풀이수/오답문제 수)

4) 프론트 최소 변경 포인트(현재 **Zustand** 구조 기준)

- `authStore`(또는 examStore에 최소 포함) 추가:
    - `authStatus: guest | authenticated`
    - `subscriptionStatus: active | inactive | canceled`
- `saveAnswer()`에서 분기:
    - guest: 로컬만
    - authenticated+active: `POST /attempts`
    - authenticated+inactive/canceled: 로컬 저장도 “초기화 정책” 때문에 저장하지 않는 쪽이 일관(권장)

5) 조대표님께 마지막 확인 **1**개만

수동 플래그 MVP에서 “유료회원 active로 전환”을 어떤 방식으로 할까요?

- A1) 관리자용 비공개 **API 1**개: `POST /admin/subscriptions/activate` (추천: 운영 편함)
- A2) DB 콘솔에서 직접 status 변경(가장 빠르지만 운영 실수 위험)

A1/A2 중 어느 쪽으로 갈지 정해주시면, 그 전제로 고팀장에게 붙여넣기용 **‘최종 지시문(백엔드 Week 0)’** 형태로 바로 작성해 드리겠습니다.